

Лабораторные работы по Linux №№1.3.7-1.3.9

«Управление пользователями и группами»

ОБЩИЕ ТРЕБОВАНИЯ
• Все действия выполнять последовательно • Каждый шаг подтверждать соответствующей командой проверки • Результаты фиксировать в отчёте • Скрипты сохранять в директории ~/lab_scripts/

ЧАСТЬ А — Базовые операции

ЗАДАНИЕ А-1
Создание организационной структуры пользователей

Сценарий:

Вы системный администратор компании "TechCorp". Необходимо создать учётные записи для нового отдела разработки.

Выполните следующее:

1. Создайте группы с указанными GID:

• dev_team	GID=2001
• qa_team	GID=2002
• devops_team	GID=2003
• management	GID=2004

2. Создайте пользователей:

Имя	UID	Основная группа	Доп. группы	Комментарий
dev_lead	1501	dev_team	management	"Lead Developer"
dev_junior	1502	dev_team	—	"Junior Developer"
qa_lead	1503	qa_team	management	"QA Lead"
qa_junior	1504	qa_team	—	"Junior QA"
devops_eng	1505	devops_team	management	"DevOps Engineer"

Все пользователи:

• Имеют домашнюю директорию
• Используют /bin/bash
• Пароль: Login@2024! (для каждого)

3. Проверьте корректность:

• Выведите содержимое /etc/group для всех созданных групп
• Выведите id для каждого пользователя
• Убедитесь, что домашние директории созданы

4. Дополнительно:

Создайте системного пользователя deploy_bot без домашней директории, без оболочки входа, комментарий "CI/CD Bot"

ЗАДАНИЕ А-2
Политики паролей

Для созданных пользователей настройте политики паролей:

Группа пользователей	Max дней	Min дней	Предупреждение	Неактивность
management (ведущие)	30	2	7	10
dev_team, qa_team	60	1	10	20
devops_team	45	1	7	14

Дополнительно:

- dev_junior должен сменить пароль при первом входе
- qa_junior – аккаунт истекает через 180 дней от сегодня
- devops_eng – заблокировать, затем разблокировать, показать содержимое строки в /etc/shadow до и после

Проверка:

- Выполните chage -l для каждого пользователя
- Убедитесь, что настройки применены корректно

ЧАСТЬ В — Bash-скрипт

ЗАДАНИЕ В-1

Скрипт аудита пользователей

Напишите скрипт user_audit.sh, который формирует полный отчёт об учётных записях системы.

СКРИПТ ДОЛЖЕН ВЫВОДИТЬ:

Раздел 1 – Общая статистика:

- Общее количество пользователей (из /etc/passwd)
- Количество системных пользователей (UID < 1000)
- Количество обычных пользователей (UID >= 1000)
- Количество заблокированных аккаунтов
- Количество аккаунтов с пустым паролем

Раздел 2 – Потенциальные угрозы безопасности:

- Пользователи с UID=0 (кроме root)
- Пользователи без пароля
- Пользователи с оболочкой /bin/bash или /bin/sh, у которых НЕТ домашней директории
- Файлы .rhosts или .netrc в домашних директориях
- Пользователи, не входившие в систему более 90 дней

Раздел 3 – Политики паролей:

- Пользователи с истёкшим паролем
- Пользователи, у которых пароль не истекает никогда (maximum days = 99999 или -1)
- Пользователи, которым нужно сменить пароль в ближайшие 7 дней

Раздел 4 – Группы:

- Пустые группы (без членов)
- Группы с дублирующимися GID
- Пользователи, состоящие в группе sudo/wheel

ТРЕБОВАНИЯ:

- Отчёт сохраняется в /var/log/user_audit_\$(date +%Y%m%d).log
- Выводится как в консоль, так и в файл (используйте tee)
- Критические находки выделяются красным цветом
- В начале отчёта: дата, имя хоста, имя запустившего

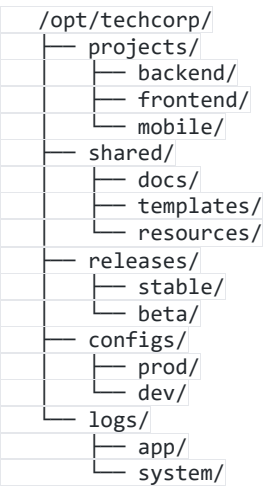
«Права доступа и владение файлами»

ЧАСТЬ А — Базовые операции

ЗАДАНИЕ А-1
Организация файловой системы проекта

Сценарий:
Настройте файловую структуру для совместной работы команды.

1. Создайте структуру директорий /opt/techcorp/:



2. Установите следующие права:

Директория	Владелец	Группа	Права	Спец.бит
/opt/techcorp/	root	dev_team	750	–
projects/backend/	dev_lead	dev_team	2770	SGID
projects/frontend/	dev_lead	dev_team	2770	SGID
projects/mobile/	dev_lead	dev_team	2770	SGID
shared/	root	dev_team	1770	Sticky
shared/docs/	dev_lead	dev_team	2755	SGID
shared/templates/	dev_lead	dev_team	2755	SGID
releases/stable/	devops_eng	devops_team	750	–
releases/beta/	devops_eng	devops_team	770	–
configs/prod/	root	devops_team	750	–
configs/dev/	devops_eng	devops_team	770	–
logs/app/	root	dev_team	1733	Sticky
logs/system/	root	root	700	–

3. Проверьте каждую директорию командой `ls -la`
Для SGID директорий: создайте тестовый файл от имени `dev_junior` и убедитесь, что файл наследует группу директории

4. Проверьте Sticky bit на `shared/`:
Войдите как `dev_junior`, создайте файл
Попробуйте удалить файл `qa_junior` – должен получить отказ
Зафиксируйте вывод ошибки в отчёте

ЗАДАНИЕ А-2
Специальные права и сценарии безопасности

1. Создайте файл `/opt/techcorp/tools/check_db.sh`
Установите права так, чтобы:

- Владелец `devops_eng` может читать, писать, выполнять
- Группа `devops_team` может только выполнять
- Остальные – нет доступа
- SUID бит установлен

После установки прав выполните:

```
ls -la /opt/techcorp/tools/check_db.sh
```

Запишите полный вывод и объясните каждый символ в строке прав

2. Найдите в системе ВСЕ файлы с SUID и SGID:

- Сохраните список в /tmp/suid_files.txt и /tmp/sgid_files.txt
- Для каждого файла выведите: путь, владельца, права
- Подсчитайте количество найденных файлов

3. Сценарий "потеря владельца":

- Создайте пользователя temp_user
- Создайте от его имени файл /tmp/temp_file.txt
- Удалите пользователя temp_user (без удаления файла)
- Найдите все "бесхозные" файлы в системе (без владельца)
- Назначьте им владельца root и группу root
- Покажите команды поиска и назначения

4. Настройте umask:

- Для пользователя dev_lead установите umask 027 постоянно
- Для пользователя qa_lead установите umask 037 постоянно
- Войдите под каждым, создайте файл и директорию
- Сравните результирующие права

«Управление процессами и сигналы»

ЧАСТЬ А — Базовые операции

ЗАДАНИЕ А-1

Исследование процессов

1. Запустите следующие процессы в фоне:

- 3 процесса: `dd if=/dev/urandom of=/dev/null`
- 2 процесса: `sleep 9999`
- 1 процесс: `tail -f /var/log/syslog`

Для каждого:

- а) Зафиксируйте PID
- б) Найдите его в `/proc/[PID]/`
- в) Прочитайте: `status`, `cmdline`, `stat`, `maps`
- г) Выведите дерево процессов через `ps tree`

2. Управление приоритетами:

- Запустите три процесса `dd` с `nice: -10, 0, +10`
- Запустите `top`, сделайте скриншот (или копию вывода)
- Измените `nice` второго процесса на `+5` прямо из `top`
- Измените `nice` третьего процесса через `renice`
- Проверьте результат через `ps -o pid,ni,comm`

3. Работа с заданиями (Job Control):

- Запустите 4 процесса `sleep` в фоне
- Выведите список `jobs`
- Приостановите `jobs 1` и `3` через `SIGSTOP` (не `Ctrl+Z`)
- Переведите `job 2` на `foreground`
- Нажмите `Ctrl+Z` (приостановите его)
- Возобновите все приостановленные задания в фоне
- Отвяжите `job 4` через `disown`
- Завершите оставшиеся задания

4. Исследование zombie-процессов:

Используя следующий скрипт (наберите вручную):

```
#!/bin/bash
```

```
# Создаём зомби-процесс
```

```
(sleep 30 && kill -0 $$ 2>/dev/null) &
```

```
CHILD_PID=$!
```

```
kill -SIGSTOP $CHILD_PID
echo "Child PID: $CHILD_PID"
```

- Найдите zombie-процесс через `ps aux` (статус Z)
- Определите его PPID
- Объясните в отчёте: почему возникают зомби и как от них избавиться

ЗАДАНИЕ A-2

Сигналы

1. Создайте тестовый скрипт `/tmp/signal_test.sh` вручную:

Скрипт должен:

- Запускать бесконечный цикл
- При получении SIGUSR1 – выводить текущее время и счётчик
- При получении SIGUSR2 – сбрасывать счётчик в 0
- При получении SIGTERM – писать "Завершение работы..." и выходить
- Игнорировать SIGINT (Ctrl+C)

Запустите скрипт в фоне и продемонстрируйте:

- а) Отправку SIGUSR1 (несколько раз)
- б) Отправку SIGUSR2 (сброс)
- в) Отправку SIGUSR1 после сброса
- г) Завершение через SIGTERM

«Systemd и управление службами»

ЧАСТЬ A — Базовые операции

ЗАДАНИЕ A-1

Создание и управление службами systemd

1. Исследуйте существующие службы:

- Выведите список всех запущенных служб
- Выведите список всех упавших служб
- Найдите службу `sshd/ssh`: посмотрите её `unit`-файл, определите зависимости (`After=`, `Wants=`, `Requires=`)
- Проверьте дерево зависимостей: `systemctl list-dependencies sshd`

2. Создайте приложение и службу для него:

Шаг 1. Создайте пользователя `webapp` (системный, без входа)

Шаг 2. Создайте директорию `/opt/webapp/` с правами для `webapp`

Шаг 3. Создайте скрипт `/opt/webapp/app.sh`:

- Записывает в лог каждые 5 секунд: время + счётчик итераций
- Обработывает SIGTERM: записывает "Получен SIGTERM" и завершается
- Обработывает SIGHUP: записывает "Перезагрузка конфигурации"
- Лог пишет в `/opt/webapp/logs/app.log`

Шаг 4. Создайте `unit`-файл `/etc/systemd/system/webapp.service`:

- Тип: `simple`
- Запуск от пользователя `webapp`
- Рабочая директория: `/opt/webapp/`
- После: `network.target`
- Автоперезапуск при падении
- Задержка перезапуска: 5 секунд
- Максимум 3 перезапуска за 60 секунд
- Лимиты: максимум 512MB RAM, максимум 50% CPU

- Логи в `journald`

Шаг 5. Активируйте и запустите службу

Шаг 6. Проверьте:

- `systemctl status webapp`
- `journalctl -u webapp -f` (в отдельном терминале)
- Отправьте `SIGUP` и посмотрите в журнале
- Принудительно завершите процесс и убедитесь в автоперезапуске

3. Создайте Override для существующей службы:

- Для службы `ssh` создайте override через `systemctl edit ssh`
- Добавьте `Environment="LOG_LEVEL=verbose"`
- Добавьте `RestartSec=10`
- Проверьте применение: `systemctl cat ssh`
- Откатите изменения

ОБЩИЕ ТРЕБОВАНИЯ К ОФОРМЛЕНИЮ ОТЧЁТА

Каждая лабораторная работа сопровождается отчётом:

1. Титульный лист (ФИО, группа, тема, дата)
2. Цель работы
3. Для каждого задания:
 - Выполненные команды (точный текст)
 - Вывод команд (скриншот или копия терминала)
 - Пояснение: что делает каждая команда/скрипт
 - Ответы на вопросы (если есть)
4. Код всех написанных скриптов с комментариями
5. Выводы по работе

Скрипты проверяются через `shellcheck` перед сдачей!

ТЕОРИЯ

1.1 Пользователи и группы в Linux

В Linux каждый пользователь имеет уникальный идентификатор **UID** (User ID), каждая группа — **GID** (Group ID).

Ключевые файлы:

Файл	Назначение
/etc/passwd	База данных пользователей
/etc/shadow	Хэши паролей
/etc/group	База данных групп
/etc/gshadow	Защищённые данные групп

Структура записи в /etc/passwd:

text

username:password:UID:GID:GECOS:home_dir:shell
student:x:1001:1001:Student User:/home/student:/bin/bash

Структура записи в /etc/shadow:

text

username:hashed_pass:lastchange:minidays:maxdays:warndays:inactive:expire
student:\$6\$hash...:19000:0:90:7:::

Тема 2.1: Создание пользователей

Команды для работы с пользователями

Bash

```
# Создание пользователя
useradd [опции] имя_пользователя

# Основные опции useradd:
# -m          создать домашнюю директорию
# -d /path     указать путь к домашней директории
# -s /bin/bash указать оболочку
# -c "Comment" добавить комментарий (GECOS)
# -u 1500      задать конкретный UID
# -g группа    указать основную группу
```

```
# -G g1,g2      указать дополнительные группы
# -e 2024-12-31 установить дату истечения аккаунта
# -r           создать системного пользователя

# Изменение пользователя
usermod [опции] имя_пользователя

# Удаление пользователя
userdel [-r] имя_пользователя # -r удаляет домашнюю директорию

# Установка/смена пароля
passwd имя_пользователя

# Просмотр информации о пользователе
id имя_пользователя
finger имя_пользователя # если установлен
getent passwd имя_пользователя
```

Практическое задание 2.1.1 (с пояснениями)

Задача: Создать пользователя `developer` с домашней директорией, `bash`-оболочкой и комментарием "Junior Developer"

Bash

```
# Шаг 1: Создаём пользователя с параметрами
sudo useradd -m -s /bin/bash -c "Junior Developer" developer

# Пояснение:
# sudo - выполнить с правами суперпользователя
# -m - создать /home/developer автоматически
# -s - установить /bin/bash как оболочку по умолчанию
# -c - добавить описание в поле GECOS

# Шаг 2: Устанавливаем пароль
sudo passwd developer
# Система запросит новый пароль дважды

# Шаг 3: Проверяем создание пользователя
id developer
# Вывод: uid=1001(developer) gid=1001(developer) groups=1001(developer)

# Шаг 4: Проверяем запись в /etc/passwd
grep developer /etc/passwd
# Вывод: developer:x:1001:1001:Junior Developer:/home/developer:/bin/bash

# Шаг 5: Проверяем домашнюю директорию
ls -la /home/developer
# Видим стандартные скрытые файлы: .bash_profile, .bashrc, .bash_logout
```


Практическое задание 2.1.2 (с пояснениями)

Задача: Создать системного пользователя `webservice` без домашней директории и без возможности входа

Bash

```
# Создаём системного пользователя
sudo useradd -r -s /usr/sbin/nologin -c "Web Service Account" webservice

# Пояснение:
# -r          - системный пользователь (UID < 1000, нет домашней директории)
# /usr/sbin/nologin - специальная оболочка, запрещающая интерактивный вход
#              при попытке входа пользователь увидит:
#              "This account is currently not available."

# Проверяем
grep webservice /etc/passwd
# webservice:x:999:999:Web Service Account:/var/webservice:/usr/sbin/nologin

# Убеждаемся, что домашняя директория не создана
ls /home/ | grep webservice
# (пустой вывод)
```

Практическое задание 2.1.3 (с пояснениями)

Задача: Изменить параметры существующего пользователя — добавить комментарий, сменить оболочку, заблокировать аккаунт

Bash

```
# Изменяем комментарий
sudo usermod -c "Senior Developer" developer

# Меняем оболочку на zsh
sudo usermod -s /bin/zsh developer

# Блокируем пользователя (добавляет ! перед хэшем в /etc/shadow)
sudo usermod -L developer

# Проверяем блокировку
sudo grep developer /etc/shadow
# developer:!!$6$hash...:... <- символ ! означает блокировку

# Разблокируем пользователя
sudo usermod -U developer

# Устанавливаем дату истечения аккаунта (через 90 дней)
sudo usermod -e $(date -d "+90 days" +%Y-%m-%d) developer

# Проверяем дату истечения
sudo chage -l developer
```

Тема 2.2: Создание и управление группами

Bash

```
# Создание группы
groupadd [опции] имя_группы
# -g GID      задать конкретный GID
# -r          создать системную группу

# Изменение группы
groupmod -n новое_имя старое_имя # переименование
groupmod -g новый_GID имя        # смена GID

# Удаление группы
groupdel имя_группы

# Добавление пользователя в группу
usermod -aG имя_группы пользователь
# -a (append) ОБЯЗАТЕЛЬНО! Без него перезапишутся все группы пользователя

# Удаление из группы
gpasswd -d пользователь группа

# Просмотр членов группы
getent group имя_группы
grep имя_группы /etc/group
```



Практическое задание 2.2.1 (с пояснениями)

Задача: Создать группы `developers` и `testers`, добавить пользователей в группы

Bash

```
# Создаём группы
sudo groupadd developers
sudo groupadd testers

# Создаём пользователей
sudo useradd -m -s /bin/bash -c "Alice Dev" alice
sudo useradd -m -s /bin/bash -c "Bob Tester" bob
sudo useradd -m -s /bin/bash -c "Carol DevOps" carol

# Устанавливаем пароли
sudo passwd alice # введите пароль
sudo passwd bob
sudo passwd carol

# Добавляем alice в группу developers
sudo usermod -aG developers alice
```

```
# Пояснение флага -aG:
# -a = append (добавить, не заменять)
# -G = supplementary groups (дополнительные группы)
# БЕЗ -a команда usermod -G developers alice
# УДАЛИТ alice из всех других групп!

# Добавляем bob в testers
sudo usermod -aG testers bob

# Добавляем carol в обе группы
sudo usermod -aG developers,testers carol

# Проверяем результат
id alice
# uid=1002(alice) gid=1002(alice) groups=1002(alice),1003(developers)

id carol
# uid=1004(carol) gid=1004(carol) groups=1004(carol),1003(developers),1004(testers)

# Просматриваем членов группы
getent group developers
# developers:x:1003:alice,carol

getent group testers
# testers:x:1004:bob,carol
```

Тема 2.3: Политики паролей

Инструмент chage — управление сроком действия паролей

Bash

```
# Просмотр политики пароля пользователя
chage -l имя_пользователя

# Опции chage:
# -m дней    минимальный срок между сменами пароля
# -M дней    максимальный срок действия пароля
# -W дней    за сколько дней предупреждать об истечении
# -I дней    период неактивности до блокировки
# -E дата    дата истечения аккаунта
# -d 0       принудительная смена пароля при следующем входе
```



Практическое задание 2.3.1 (с пояснениями)

Задача: Настроить политику паролей для пользователя `developer`

Bash

```
# Устанавливаем политику:
# - минимальный возраст пароля: 1 день (нельзя менять чаще)
# - максимальный возраст: 90 дней
# - предупреждение за 14 дней
# - блокировка через 30 дней после истечения
sudo chage -m 1 -M 90 -W 14 -I 30 developer

# Проверяем применённые настройки
sudo chage -l developer
# Last password change           : Dec 01, 2024
# Password expires                : Mar 01, 2025   (через 90 дней)
# Password inactive              : Mar 31, 2025   (через 30 дней после)
# Account expires                : never
# Minimum number of days between pw change : 1
# Maximum number of days between pw change : 90
# Number of days of warning before pw expires : 14

# Принудительно требуем смену пароля при следующем входе
sudo chage -d 0 developer
# Это устанавливает дату последней смены в 01.01.1970
# При следующем входе система потребует сменить пароль
```



Практическое задание 2.3.2 (с пояснениями)

Задача: Настроить системные политики паролей через PAM и /etc/login.defs

Bash

```
# Просматриваем глобальные настройки паролей
cat /etc/login.defs | grep -E "PASS_|UID_|GID_"

# Основные параметры в /etc/login.defs:
# PASS_MAX_DAYS 90 - максимальный срок пароля
# PASS_MIN_DAYS 1 - минимальный срок пароля
# PASS_MIN_LEN 8 - минимальная длина (устаревший параметр)
# PASS_WARN_AGE 14 - предупреждение об истечении

# Редактируем глобальные настройки
sudo nano /etc/login.defs
# Изменяем:
# PASS_MAX_DAYS 90
# PASS_MIN_DAYS 1
# PASS_WARN_AGE 14

# ВАЖНО: эти настройки применяются только к НОВЫМ пользователям!
# Для существующих используем chage

# Настройка сложности паролей через PAM (pam_pwquality)
# Файл настроек: /etc/security/pwquality.conf

sudo nano /etc/security/pwquality.conf

# Рекомендуемые настройки:
# minlen = 12 # минимальная длина
```

```
# dcredit = -1      # минимум 1 цифра (-N = минимум N символов)
# ucredit = -1      # минимум 1 заглавная буква
# lcredit = -1      # минимум 1 строчная буква
# ocredit = -1      # минимум 1 спецсимвол
# maxrepeat = 3     # не более 3 одинаковых символов подряд
# usercheck = 1     # запрет использования имени пользователя в пароле
# retry = 3         # 3 попытки при вводе пароля

# Проверяем, подключён ли модуль pam_pwquality
grep pwquality /etc/pam.d/common-password
# password requisite pam_pwquality.so retry=3
```

✓ Самостоятельные задания — Блок 1 (без подсказок)

text

САМОСТОЯТЕЛЬНЫЕ ЗАДАНИЯ – БЛОК 1 Пользователи, группы, политики паролей

Задание 1.1

Создайте пользователя "intern" с UID=2000, домашней директорией /home/intern_workspace, оболочкой /bin/bash и комментарием "Company Intern". Установите пароль "Intern@2024!". Проверьте, что все параметры применились корректно.

Задание 1.2

Создайте группу "hr_department" с GID=3000.
Создайте пользователей: hr_alice, hr_bob, hr_manager.
Добавьте всех трёх в группу hr_department.
Убедитесь, что hr_manager также состоит в группе sudo/wheel.

Задание 1.3

Настройте политику паролей для пользователя hr_manager:

- Пароль должен меняться каждые 60 дней
- Нельзя менять пароль чаще чем раз в 2 дня
- Предупреждение за 10 дней
- Блокировка через 15 дней после истечения
- При следующем входе потребовать смену пароля

Задание 1.4

Найдите в системе всех пользователей с UID больше 1000 (реальных пользователей, не системных).
Выведите их имена, домашние директории и оболочки.
Подсказка: используйте awk для разбора /etc/passwd

Задание 1.5

Заблокируйте пользователя hr_bob, проверьте блокировку, затем разблокируйте его. Покажите содержимое /etc/shadow до и после каждой операции (только строку hr_bob).

ЧАСТЬ 3: ПРАВА ДОСТУПА К ФАЙЛАМ И ДИРЕКТОРИЯМ

3.1 Теория прав доступа

Структура прав доступа

text

```
-rwxr-xr-- 1 alice developers 4096 Dec 1 12:00 script.sh
```

| H H H H

| | | | Права остальных (others): r-- = 4

| | | Права группы (group): r-x = 5

| | Права владельца (user): rwx = 7

| Тип: - файл, d директория, l символ.ссылка

Обозначения прав:

r = read (чтение) = 4

w = write (запись) = 2

x = execute (выполнение) = 1

- = нет права = 0

Для ДИРЕКТОРИЙ:

r = можно просматривать содержимое (ls)

w = можно создавать/удалять файлы внутри

x = можно входить в директорию (cd) и обращаться к файлам

Специальные биты

text

SUID (Set User ID) = 4000

└─ На исполняемом файле: запускается с правами владельца файла

└─ Пример: /usr/bin/passwd (владелец root, но запускают обычные юзеры)

SGID (Set Group ID) = 2000

└─ На файле: запускается с правами группы файла

└─ На директории: новые файлы наследуют группу директории

Sticky Bit = 1000

└─ На директории: удалять файлы может только владелец файла

└─ Пример: /tmp (все пишут, но удаляют только свои файлы)

Обозначение в ls:

-rwsr-xr-x SUID установлен (s вместо x у владельца)

-rwxr-sr-x SGID установлен (s вместо x у группы)

drwxrwxrwt Sticky bit установлен (t вместо x у others)

Практическое задание 3.1.1 (с пояснениями)

Задача: Создать структуру директорий для проекта и настроить права

Bash

```
# Создаём структуру проекта
sudo mkdir -p /opt/project/{src,docs,logs,config,backup}

# Смотрим текущие права (созданы с правами по умолчанию)
ls -la /opt/project/

# Устанавливаем владельца и группу
sudo chown root:developers /opt/project
sudo chown root:developers /opt/project/{src,docs,logs,config,backup}

# Устанавливаем права на основную директорию проекта
# 750 = rwxr-x---
# Владелец (root): rwx - полный доступ
# Группа (developers): r-x - чтение и вход
# Остальные: --- - нет доступа
sudo chmod 750 /opt/project

# Права на src - разработчики могут читать и писать
# 770 = rwxrwx---
sudo chmod 770 /opt/project/src

# Права на docs - разработчики только читают
# 750 = rwxr-x---
sudo chmod 750 /opt/project/docs

# Права на logs - только root пишет, группа читает
# 750 = rwxr-x---
sudo chmod 750 /opt/project/logs

# Права на config - строго для root
# 700 = rwx-----
sudo chmod 700 /opt/project/config

# Права на backup - только root
# 700 = rwx-----
sudo chmod 700 /opt/project/backup

# Проверяем результат
ls -la /opt/project/
# drwxr-x--- root developers /opt/project/
# drwxrwx--- root developers src/
# drwxr-x--- root developers docs/
# drwxr-x--- root developers logs/
# drwx----- root root config/
# drwx----- root root backup/
```



Практическое задание 3.1.2 (с пояснениями)

Задача: Символьная запись прав chmod

Bash

```
# Символьная форма chmod:
# u = user (владелец)
# g = group (группа)
# o = others (остальные)
# a = all (все)
# + добавить право
# - убрать право
# = установить точно

# Создаём тестовые файлы
touch /tmp/test_file.sh
touch /tmp/test_data.txt
mkdir /tmp/test_dir

# Добавляем право выполнения для владельца
chmod u+x /tmp/test_file.sh
ls -la /tmp/test_file.sh
# -rwxr--r-- (было -rw-rw-r--)

# Убираем право записи у группы и остальных
chmod go-w /tmp/test_data.txt
ls -la /tmp/test_data.txt
# -rw-r--r--

# Устанавливаем точные права: владелец rwx, группа r-x, остальные ---
chmod u=rwx,g=rx,o= /tmp/test_file.sh
ls -la /tmp/test_file.sh
# -rwxr-x---

# Рекурсивное применение прав
# ОСТОРОЖНО: файлы и директории получат одинаковые права
# что обычно не желательно (директориям нужен бит x)
chmod -R 755 /tmp/test_dir

# Правильный подход - разные права для файлов и директорий:
find /tmp/test_dir -type d -exec chmod 755 {} \; # директориям
find /tmp/test_dir -type f -exec chmod 644 {} \; # файлам
find /tmp/test_dir -name "*.sh" -exec chmod 755 {} \; # скриптам

# Установка SUID на файл
chmod u+s /tmp/test_file.sh
ls -la /tmp/test_file.sh
# -rwsr-x--- (s вместо x у владельца)

# Установка Sticky bit на директорию /tmp (уже установлен)
ls -la /tmp
# drwxrwxrwt (t в конце)

# Установка SGID на директорию проекта
sudo chmod g+s /opt/project/src
ls -la /opt/project/
# drwxrwsr-- src/ (s в правах группы)
# Теперь все новые файлы в src/ будут наследовать группу 'developers'
```


3.2 Изменение владельца файлов

Bash

```
# Смена владельца
chown пользователь файл
chown пользователь:группа файл
chown :группа файл # только группу

# Рекурсивно
chown -R пользователь:группа директория

# Только группа (аналог chown :группа)
chgrp группа файл
chgrp -R группа директория
```



Практическое задание 3.2.1 (с пояснениями)

Bash

```
# Создаём файлы для демонстрации
sudo touch /opt/project/src/main.py
sudo touch /opt/project/src/utils.py
sudo touch /opt/project/docs/README.md

# Смотрим текущего владельца
ls -la /opt/project/src/
# -rw-r--r-- 1 root root ... main.py
# -rw-r--r-- 1 root root ... utils.py

# Меняем владельца main.py на alice
sudo chown alice /opt/project/src/main.py
ls -la /opt/project/src/main.py
# -rw-r--r-- 1 alice root ... main.py

# Меняем владельца И группу utils.py
sudo chown alice:developers /opt/project/src/utils.py
ls -la /opt/project/src/utils.py
# -rw-r--r-- 1 alice developers ... utils.py

# Меняем только группу README.md
sudo chgrp developers /opt/project/docs/README.md
ls -la /opt/project/docs/README.md
# -rw-r--r-- 1 root developers ... README.md

# Рекурсивная смена владельца всего проекта
sudo chown -R root:developers /opt/project/
ls -la /opt/project/src/
# Все файлы теперь принадлежат root:developers

# Практический кейс: передача проекта от одного разработчика другому
# Находим все файлы, принадлежащие alice, и меняем владельца на carol
sudo find /opt/project -user alice -exec chown carol {} \;
```

3.3 Установка прав по умолчанию — umask

Как работает umask

text

umask определяет права, которые будут ВЫЧТЕНЫ из максимальных:

Для файлов: 666 (максимум без бита выполнения)
Для директорий: 777

При umask=022:

Файлы: 666 - 022 = 644 (rw-r--r--)
Директории: 777 - 022 = 755 (rwxr-xr-x)

При umask=027:

Файлы: 666 - 027 = 640 (rw-r-----)
Директории: 777 - 027 = 750 (rwxr-x---

При umask=077:

Файлы: 666 - 077 = 600 (rw-----)
Директории: 777 - 077 = 700 (rwx-----)



Практическое задание 3.3.1 (с пояснениями)

Bash

```
# Просматриваем текущую umask
umask
# 0022

# Просматриваем в символьной форме
umask -S
# u=rwx,g=rx,o=rx

# Демонстрация работы umask
# Текущая umask=022
touch /tmp/default_file
mkdir /tmp/default_dir
ls -la /tmp/default_file # -rw-r--r-- (644)
ls -ld /tmp/default_dir # drwxr-xr-x (755)

# Меняем umask для текущей сессии на 027
umask 027
touch /tmp/restricted_file
mkdir /tmp/restricted_dir
ls -la /tmp/restricted_file # -rw-r----- (640)
ls -ld /tmp/restricted_dir # drwxr-x--- (750)
```

```
# Меняем umask на максимально ограничительную 077
umask 077
touch /tmp/private_file
mkdir /tmp/private_dir
ls -la /tmp/private_file # -rw----- (600)
ls -ld /tmp/private_dir  # drwx----- (700)

# Возвращаем стандартную umask
umask 022

# Установка umask ПОСТОЯННО для конкретного пользователя
# Добавляем в ~/.bashrc или ~/.bash_profile:
echo "umask 027" >> ~/.bashrc
source ~/.bashrc

# Установка umask СИСТЕМНО для всех пользователей
# В файле /etc/profile или /etc/bash.bashrc:
sudo nano /etc/profile
# Добавляем строку: umask 022

# Проверка текущей umask в числовом виде
umask
```

✓ Самостоятельные задания — Блок 2 (без подсказок)

text

САМОСТОЯТЕЛЬНЫЕ ЗАДАНИЯ — БЛОК 2

Права доступа, владелец, umask

Задание 2.1

Создайте директорию /srv/shared с правами:

- Владелец (root): полный доступ
- Группа (developers): чтение и выполнение (вход)
- Остальные: нет доступа

Установите SGID бит, чтобы все новые файлы наследовали группу developers. Проверьте работу SGID, создав файл от имени пользователя alice (должна быть в группе developers).

Задание 2.2

Найдите все файлы в /usr/bin с установленным SUID битом. Для трёх любых найденных файлов объясните (в комментарии к команде), ЗАЧЕМ им нужен SUID.

Задание 2.3

Создайте директорию /tmp/team_folder с правами 1777. Создайте в ней файлы от имени разных пользователей. Проверьте, что пользователь alice НЕ МОЖЕТ удалить файл, принадлежащий bob. Объясните, как работает sticky bit.

Задание 2.4

Настройте umask=027 для пользователя hr_manager постоянно. Войдите под этим пользователем (su - hr_manager) и создайте несколько файлов и директорий. Убедитесь, что права применяются корректно.

Задание 2.5

Напишите скрипт `setup_project.sh`, который:

1. Создаёт структуру: `/opt/myproject/{src,tests,docs,logs}`
2. Создаёт группу `"myproject_team"`
3. Устанавливает правильные права на каждую директорию
4. Устанавливает SGID на `src` и `tests`
5. Выводит итоговую структуру командой `ls -laR`

ЧАСТЬ 4: УПРАВЛЕНИЕ ПРОЦЕССАМИ

4.1 Основы управления процессами

Ключевые концепции

text

PID - уникальный идентификатор процесса

PPID - PID родительского процесса

UID - пользователь, запустивший процесс

State - состояние процесса:

R - Running (выполняется или готов к выполнению)

S - Sleeping (ожидает события, прерываемый сон)

D - Uninterruptible sleep (ожидает I/O, нельзя прервать)

Z - Zombie (завершился, но родитель не считал код возврата)

T - Stopped (остановлен сигналом SIGSTOP/SIGTSTP)

I - Idle kernel thread

Priority и Nice:

nice: от -20 (высший приоритет) до 19 (низший)

По умолчанию: 0

Только root может устанавливать отрицательные значения nice



Практическое задание 4.1.1 (с пояснениями)

Задача: Изучить инструменты просмотра процессов

Bash

ps - статический снимок процессов

Показать процессы текущего терминала
`ps`

Показать все процессы всех пользователей (BSD-стиль)

`ps aux`

# USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
# root	1	0.0	0.1	16952	1824	?	Ss	08:00	0:01	/sbin/init

Пояснение колонок:

USER - владелец процесса

PID - идентификатор

%CPU - использование CPU

%MEM - использование памяти

VSZ - виртуальный размер (KB)

RSS - реальная память (KB)

TTY - терминал (? = нет терминала)

STAT - статус (R,S,D,Z,T + флаги)

START - время запуска

TIME - суммарное время CPU

COMMAND - команда

Показать в виде дерева (видны родительские связи)

`ps auxf`

или

`pstree -p` # более наглядное дерево

Показать процессы конкретного пользователя

`ps -u alice`

`ps -U alice` # по реальному UID

Показать конкретный процесс по PID

`ps -p 1234`

Найти PID по имени процесса

`pgrep bash`

`pgrep -l bash` # с именем

`pgrep -a bash` # с полной командой

Найти PID через pidof

`pidof bash`

Детальная информация через /proc

`ls /proc/$$` # \$\$ = PID текущего shell

`cat /proc/$$/status` # детальный статус

`cat /proc/$$/cmdline` # команда запуска

`cat /proc/$$/environ` # переменные окружения

4.2 Работа с фоновыми процессами

text

Управление заданиями (Job Control):

Передний план (Foreground) - процесс, взаимодействующий с терминалом

Фоновый режим (Background) - процесс, работающий без доступа к терминалу

Команды управления:

& - запустить команду в фоне
Ctrl+Z - приостановить текущий процесс (SIGTSTP)
Ctrl+C - завершить текущий процесс (SIGINT)
jobs - список фоновых/приостановленных заданий
fg [%N] - перевести задание N на передний план
bg [%N] - продолжить выполнение задания N в фоне
disown - отвязать задание от терминала
nohup - запустить процесс, устойчивый к закрытию терминала



Практическое задание 4.2.1 (с пояснениями)

Задача: Управление фоновыми процессами

Bash

```
# Запускаем длительный процесс в фоне
sleep 300 &
# [1] 12345 <- [номер_задания] PID

# Запускаем ещё несколько
sleep 400 &
sleep 500 &

# Просматриваем все задания
jobs
# [1]   Running    sleep 300 &
# [2]   Running    sleep 400 &
# [3]-  Running    sleep 500 &
# Пояснение: + текущее задание, - предыдущее

jobs -l # с PID
# [1]  12345 Running    sleep 300 &
# [2]  12346 Running    sleep 400 &

# Переводим задание 2 на передний план
fg %2
# sleep 400 (выполняется в foreground)

# Приостанавливаем его: нажимаем Ctrl+Z
# [2]+  Stopped    sleep 400

# Проверяем статус
jobs
# [1]   Running    sleep 300 &
# [2]+  Stopped    sleep 400
# [3]-  Running    sleep 500 &

# Возобновляем задание 2 в фоне
bg %2
# [2]+  sleep 400 &

# Отвязываем задание 1 от терминала
# (при закрытии терминала оно продолжит работу)
disown %1

# Или используем nohup для запуска с нуля
```

```
nohup sleep 600 &
# nohup: игнорирует сигнал SIGHUP
# Вывод перенаправляется в ./nohup.out

# Убиваем все задания sleep
kill %1 %2 %3
# или
killall sleep

# Более устойчивый способ запуска фоновых процессов
# Использование screen или tmux
screen -S mysession # создать сессию
# ... запустить команды ...
# Ctrl+A, D - отсоединиться
screen -r mysession # переподключиться
screen -ls          # список сессий
```

Практическое задание 4.2.2 (с пояснениями)

Задача: Управление приоритетом процессов

Bash

```
# Запускаем процесс с низким приоритетом (nice=10)
nice -n 10 sleep 300 &
# Процессу дают меньше CPU при нехватке ресурсов

# Запускаем с высоким приоритетом (только root)
sudo nice -n -10 sleep 400 &

# Проверяем nice значения
ps -o pid,ni,comm
# PID  NI  COMMAND
# 12345 10  sleep
# 12346 -10  sleep

# Изменяем приоритет ЗАПУЩЕННОГО процесса
# renice [новое_nice] [опции] PID

# Понижаем приоритет (любой пользователь)
renice 15 -p 12345

# Повышаем приоритет (только root)
sudo renice -5 -p 12345

# Изменяем приоритет всем процессам пользователя
sudo renice 10 -u alice

# Изменяем приоритет всем процессам группы
sudo renice 10 -g developers

# Проверяем через ps
ps -o pid,ni,user,comm -p 12345
```

4.3 Сигналы для управления процессами

Таблица основных сигналов

text

№	Название	Описание	Действие по умолчанию
1	SIGHUP	Отключение терминала	Завершение
2	SIGINT	Прерывание (Ctrl+C)	Завершение
3	SIGQUIT	Выход (Ctrl+\)	Завершение+дамп
9	SIGKILL	Принудительное завершение	Завершение (нельзя перехватить)
10	SIGUSR1	Пользовательский сигнал 1	Завершение
11	SIGSEGV	Ошибка сегментации	Завершение+дамп
12	SIGUSR2	Пользовательский сигнал 2	Завершение
15	SIGTERM	Запрос завершения (по умолчанию)	Завершение (можно перехватить)
17	SIGCHLD	Изменение состояния дочернего	Игнорирование
18	SIGCONT	Продолжить выполнение	Продолжение
19	SIGSTOP	Остановить (нельзя перехватить)	Остановка
20	SIGTSTP	Пауза (Ctrl+Z)	Остановка (можно перехватить)

Практическое задание 4.3.1 (с пояснениями)

Задача: Отправка сигналов процессам

Bash

```
# Подготовка: запускаем тестовый процесс
sleep 1000 &
MYPID=$! # сохраняем PID последнего фонового процесса
echo "PID процесса: $MYPID"

# СПОСОБЫ ОТПРАВКИ СИГНАЛОВ:

# 1. Команда kill (отправляет сигналы по PID)
# По умолчанию отправляет SIGTERM (15)
kill $MYPID

# Отправить конкретный сигнал по номеру
kill -9 $MYPID # SIGKILL
kill -15 $MYPID # SIGTERM
kill -SIGKILL $MYPID # по имени

# 2. Команда pkill (по имени процесса)
sleep 1000 &
pkill sleep # SIGTERM всем процессам sleep
pkill -9 sleep # SIGKILL
pkill -u alice sleep # только процессы пользователя alice

# 3. Команда killall (по точному имени)
sleep 1000 &
sleep 2000 &
killall sleep # завершить все процессы с именем sleep
killall -9 sleep # принудительно
```



```
# Практический пример: правильная последовательность завершения
# 1. Сначала вежливо просим завершиться (SIGTERM)
kill -15 $MYPID
sleep 3 # даём 3 секунды на корректное завершение

# 2. Проверяем, завершился ли
if ps -p $MYPID > /dev/null 2>&1; then
    echo "Процесс ещё работает, применяем SIGKILL"
    kill -9 $MYPID
else
    echo "Процесс завершён корректно"
fi

# Отправка SIGHUP (часто используется для перезагрузки конфигурации)
# Nginx, sshd и другие демоны читают конфиг заново при получении SIGHUP
sudo kill -HUP $(pgrep nginx)
# или
sudo nginx -s reload # для nginx это обёртка над SIGHUP

# Остановка и продолжение процесса
sleep 300 &
MYPID=$!
kill -SIGSTOP $MYPID # остановить
ps -o pid,stat -p $MYPID # STAT покажет T (stopped)
kill -SIGCONT $MYPID # продолжить
ps -o pid,stat -p $MYPID # STAT покажет S (sleeping)

# Просмотр всех доступных сигналов
kill -l
```

4.4 Утилита top — мониторинг в реальном времени

Интерфейс top

text

```
top - 14:32:15 up 2:15, 2 users, load average: 0.15, 0.12, 0.10
Tasks: 156 total, 1 running, 155 sleeping, 0 stopped, 0 zombie
%Cpu(s): 2.3 us, 0.7 sy, 0.0 ni, 96.7 id, 0.3 wa, 0.0 hi, 0.0 si
MiB Mem : 7856.4 total, 2341.2 free, 3214.8 used, 2300.4 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 4120.3 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1234	alice	20	0	156748	12345	9876	R	2.3	0.2	0:01.23	python3
5678	root	20	0	45678	4567	3456	S	0.3	0.1	0:00.45	sshd

Строка CPU:

us = user space (пользовательские процессы)

sy = system (ядро)

ni = nice (процессы с измененным приоритетом)

id = idle (простой)

wa = wait (ожидание I/O) <- если высокий, проблема с дисками

hi = hardware interrupts

si = software interrupts



Практическое задание 4.4.1 (с пояснениями)

Задача: Работа с утилитой top в интерактивном режиме

Bash

```
# Запускаем top
top

# ИНТЕРАКТИВНЫЕ КОМАНДЫ В TOP:

# Сортировка:
# P - по CPU (по умолчанию)
# M - по памяти
# N - по PID
# T - по времени CPU
# R - инвертировать сортировку

# Отображение:
# 1 - показать все CPU ядра отдельно
# m - переключить отображение памяти (числа/графики)
# t - переключить отображение задач (числа/графики)

# Управление процессами прямо из top:
# k - убить процесс (запросит PID и сигнал)
# r - изменить nice (renice)

# Фильтрация:
# u - фильтр по пользователю (введите имя)
# o - добавить фильтр (например: COMMAND=python)

# Прочее:
# d или s - изменить интервал обновления
# H - показать потоки (threads)
# c - показать полную командную строку
# q - выход
# h - справка

# Запуск с параметрами:
top -u alice          # только процессы alice
top -p 1234,5678      # только конкретные PID
top -b -n 3           # batch режим, 3 итерации (для скриптов)
top -b -n 1 > /tmp/top_snapshot.txt # сохранить снимок

# htop - улучшенная версия top (если установлен)
# sudo apt install htop / sudo yum install htop
htop
# Преимущества: цвета, мышь, горизонтальная/вертикальная прокрутка
# F2 - настройки
# F3 - поиск
# F5 - дерево процессов
# F6 - сортировка
# F9 - отправить сигнал

# Пример: найти самые "тяжёлые" процессы
# Запустить несколько процессов для демонстрации
yes > /dev/null & # 100% CPU
yes > /dev/null &
jobs # видим запущенные процессы
top # смотрим их в top, нажимаем P для сортировки по CPU
# Завершаем
killall yes
```

4.5 Systemd — управление службами

Основы systemd

text

systemd - система инициализации и управления службами

Unit файлы находятся в:

/lib/systemd/system/	- файлы от пакетов (не редактировать)
/etc/systemd/system/	- пользовательские/переопределения
/run/systemd/system/	- временные (создаются в runtime)

Типы units:

.service	- службы (демоны)
.socket	- сокет
.timer	- задачи по расписанию (аналог cron)
.target	- группы units (аналог runlevel)
.mount	- точки монтирования
.path	- мониторинг файловой системы



Практическое задание 4.5.1 (с пояснениями)

Задача: Управление службами через systemctl

Bash

ОСНОВНЫЕ КОМАНДЫ SYSTEMCTL:

Статус службы

`sudo systemctl status sshd`

• ssh.service - OpenBSD Secure Shell server

Loaded: loaded (/lib/systemd/system/ssh.service; enabled)

Active: active (running) since ...

Process: 1234 ExecStart=/usr/sbin/sshd -D

Main PID: 1234 (sshd)

CGroup: /system.slice/ssh.service

└─1234 /usr/sbin/sshd -D

Статус показывает:

Loaded - загружена ли служба (enabled/disabled - автозапуск)

Active - текущее состояние (active running / inactive dead / failed)

Запуск и остановка

`sudo systemctl start nginx`

`sudo systemctl stop nginx`

`sudo systemctl restart nginx` # стоп + старт

`sudo systemctl reload nginx` # перезагрузить конфиг (без остановки)

Автозапуск

```

sudo systemctl enable nginx      # включить автозапуск
sudo systemctl disable nginx     # выключить автозапуск
sudo systemctl enable --now nginx # включить + запустить прямо сейчас
sudo systemctl disable --now nginx # выключить + остановить

# Просмотр всех служб
systemctl list-units --type=service
systemctl list-units --type=service --state=running # только запущенные
systemctl list-units --type=service --state=failed  # только упавшие

# Список всех unit файлов
systemctl list-unit-files --type=service

# Перезагрузка конфигурации systemd (после изменения unit файлов)
sudo systemctl daemon-reload

# Создание собственной службы:
sudo nano /etc/systemd/system/myapp.service

# Содержимое unit файла:
cat << 'EOF' | sudo tee /etc/systemd/system/myapp.service
[Unit]
Description=My Application Service
After=network.target      # запускать после сети
Wants=network.target      # желательна сеть (не обязательна)

[Service]
Type=simple              # простой процесс
User=myapp               # от какого пользователя
Group=myapp
WorkingDirectory=/opt/myapp # рабочая директория
ExecStart=/opt/myapp/start.sh # команда запуска
ExecStop=/opt/myapp/stop.sh # команда остановки (опционально)
Restart=on-failure        # перезапускать при ошибке
RestartSec=5              # через 5 секунд
StandardOutput=journal    # логи в journald
StandardError=journal

[Install]
WantedBy=multi-user.target # в каком target запускаться
EOF

# Применяем изменения
sudo systemctl daemon-reload

# Запускаем и проверяем
sudo systemctl start myapp
sudo systemctl status myapp
sudo systemctl enable myapp # автозапуск

```



Практическое задание 4.5.2 (с пояснениями)

Задача: Работа с systemd targets (уровни запуска)

Bash

```

# Просмотр текущего target
systemctl get-default
# multi-user.target (обычно для серверов)

```

```
# graphical.target (для десктопов с GUI)

# Список всех targets
systemctl list-units --type=target

# Соответствие targets и runlevels:
# runlevel 0 = poweroff.target (выключение)
# runlevel 1 = rescue.target (восстановительный режим)
# runlevel 3 = multi-user.target (консольный многопользовательский)
# runlevel 5 = graphical.target (с GUI)
# runlevel 6 = reboot.target (перезагрузка)

# Изменение target на лету
sudo systemctl isolate multi-user.target # переключиться немедленно
sudo systemctl isolate graphical.target

# Изменение target по умолчанию
sudo systemctl set-default multi-user.target

# Аварийный режим
sudo systemctl rescue # однопользовательский режим
sudo systemctl emergency # аварийный (монтирует только root)

# Перезагрузка и выключение через systemd
sudo systemctl reboot
sudo systemctl poweroff
sudo systemctl halt
sudo systemctl suspend # спящий режим
```

Работа с логами — journalctl и syslog

Структура системы логирования

Источники логов:

Приложения → rsyslog/syslog-ng → /var/log/*.log
systemd units → journald → /run/log/journal/
Ядро (kernel) → journald + /var/log/kern.log

Уровни важности (severity):

```
0 - emerg (система неработоспособна)
1 - alert (требуется немедленное вмешательство)
2 - crit (критическая ошибка)
3 - err (ошибка)
4 - warning (предупреждение)
5 - notice (важное уведомление)
6 - info (информация)
7 - debug (отладка)
```

Важные файлы логов:

```
/var/log/syslog или /var/log/messages - основной лог
/var/log/auth.log или /var/log/secure - авторизация
/var/log/kern.log - ядро
/var/log/dpkg.log или /var/log/yum.log - пакетный менеджер
/var/log/nginx/access.log - доступ к nginx
/var/log/nginx/error.log - ошибки nginx
```

Задача: Работа с journalctl

Bash

```
# Просмотр всех логов (от старых к новым)
journalctl

# Логи с самых новых (в конце)
journalctl -e # jump to end

# Следить за логами в реальном времени (как tail -f)
journalctl -f

# Логи конкретной службы
journalctl -u nginx
journalctl -u nginx -f # в реальном времени

# Логи за период времени
journalctl --since "2024-01-01 00:00:00"
journalctl --until "2024-01-31 23:59:59"
journalctl --since "2024-01-01" --until "2024-01-31"
journalctl --since "1 hour ago"
journalctl --since "yesterday"
journalctl --since today

# Фильтрация по приоритету
journalctl -p err # ошибки и выше
journalctl -p warning # предупреждения и выше
journalctl -p debug # все уровни
journalctl -p 3 # по числу (3=err)
journalctl -p err -u nginx # ошибки конкретной службы

# Фильтрация по пользователю
journalctl _UID=1001 # по UID
journalctl _COMM=sshd # по имени процесса

# Количество строк
journalctl -n 50 # последние 50 строк
journalctl -n 100 -u sshd # последние 100 строк sshd

# Форматы вывода
journalctl -o json # JSON формат
journalctl -o json-pretty # красивый JSON
journalctl -o short # стандартный (по умолчанию)
journalctl -o verbose # все поля
journalctl -o cat # только сообщение (без метаданных)

# Информация о хранилище логов
journalctl --disk-usage # размер логов на диске
journalctl --verify # проверка целостности
journalctl --list-boots # список загрузок системы
journalctl -b # логи текущей загрузки
journalctl -b -1 # логи предыдущей загрузки
journalctl -b -2 # логи позапрошлой загрузки

# Очистка логов
sudo journalctl --vacuum-size=500M # оставить не более 500MB
sudo journalctl --vacuum-time=30d # оставить не более 30 дней
sudo journalctl --vacuum-files=10 # оставить не более 10 файлов

# Комплексный пример: найти все ошибки ssh за последние 24 часа
journalctl -u ssh -p err --since "24 hours ago"
```

```
# Просмотр основного лога
sudo tail /var/log/syslog          # последние 10 строк
sudo tail -f /var/log/syslog      # в реальном времени
sudo tail -n 100 /var/log/syslog  # последние 100 строк
sudo head -n 50 /var/log/syslog   # первые 50 строк

# Просмотр лога авторизации (попытки входа, sudo и т.д.)
sudo tail -f /var/log/auth.log
# Debian/Ubuntu: /var/log/auth.log
# CentOS/RHEL: /var/log/secure

# Поиск в логах с grep
sudo grep "Failed password" /var/log/auth.log # неудачные попытки входа
sudo grep "sshd" /var/log/auth.log | tail -20
sudo grep -i "error" /var/log/syslog | grep -v "dns" # ошибки, кроме dns

# Подсчёт событий
sudo grep "Failed password" /var/log/auth.log | wc -l

# Поиск атак методом перебора:
sudo grep "Failed password" /var/log/auth.log | \
    awk '{print $11}' | \
    sort | uniq -c | sort -rn | head -20
# Показывает IP-адреса с наибольшим числом неудачных попыток

# Просмотр логов с указанием имён файлов (когда несколько файлов)
sudo tail -f /var/log/nginx/access.log /var/log/nginx/error.log

# Использование less для удобного просмотра
sudo less /var/log/syslog
# В less:
# G - перейти в конец
# g - перейти в начало
# / - поиск вперёд
# ? - поиск назад
# n - следующее совпадение
# N - предыдущее совпадение
# q - выход

# Ротация логов - logrotate
cat /etc/logrotate.conf          # основная конфигурация
ls /etc/logrotate.d/             # конфигурации для отдельных приложений
cat /etc/logrotate.d/nginx      # пример конфигурации nginx

# Пример конфигурации logrotate:
cat << 'EOF'
/var/log/myapp/*.log {
    daily          # ротация ежедневно
    rotate 30      # хранить 30 файлов
    compress       # сжимать старые логи (gzip)
    delaycompress  # сжимать не сразу (со следующего цикла)
    missingok      # не ошибаться, если файл отсутствует
    notifempty     # не ротировать пустые файлы
    create 640 myapp myapp # создавать новый файл с правами 640
    postrotate
        systemctl reload myapp # перезагрузить приложение после ротации
    endscript
}
EOF

# Принудительная ротация (для тестирования)
sudo logrotate -f /etc/logrotate.d/nginx

# Запись в syslog из скрипта
logger "My application started"
```

```
logger -p user.err "My application error occurred"
logger -t myapp "Custom tag message"
# Проверяем
sudo grep "myapp" /var/log/syslog
```

Практическое задание 4.6.3 (с пояснениями)

Задача: Настройка rsyslog для пользовательского логирования

Bash

```
# Просматриваем основную конфигурацию rsyslog
cat /etc/rsyslog.conf

# Создаём правило для логирования в отдельный файл
sudo nano /etc/rsyslog.d/50-myapp.conf

# Содержимое файла:
cat << 'EOF' | sudo tee /etc/rsyslog.d/50-myapp.conf
# Логировать все сообщения с тегом myapp в отдельный файл
if $programname == 'myapp' then /var/log/myapp.log
& stop # не передавать дальше в основной лог
EOF

# Перезапускаем rsyslog
sudo systemctl restart rsyslog

# Тестируем
logger -t myapp "Test message 1"
logger -t myapp -p user.err "Test error message"

# Проверяем
cat /var/log/myapp.log
# Dec 01 14:32:01 hostname myapp: Test message 1
# Dec 01 14:32:01 hostname myapp: Test error message
```

Самостоятельные задания — Блок 3 (без подсказок)

text

САМОСТОЯТЕЛЬНЫЕ ЗАДАНИЯ — БЛОК 3
Процессы, сигналы, systemd, логи

Задание 3.1 — Управление процессами
Запустите 5 процессов "sleep" разной длительности в фоновом

режиме. Приостановите 2-й и 4-й процессы. Переведите 3-й на передний план, затем снова отправьте в фон. Завершите все нечётные задания через kill, чётные через killall. Продемонстрируйте каждый шаг командой jobs.

Задание 3.2 – Приоритеты процессов

Запустите три процесса "dd if=/dev/urandom of=/dev/null" с nice значениями: -5, 0, +10 (для -5 нужны права root). Запустите top и, не завершая его, объясните в отчёте:

- Как распределяется CPU между процессами?
- Что означают колонки PR и NI в top?
- Как изменить nice прямо из top?

Завершите все три процесса.

Задание 3.3 – Сигналы

Напишите bash-скрипт "signal_demo.sh", который:

1. Запускает бесконечный цикл с выводом счётчика каждую секунду
2. Перехватывает SIGTERM и выводит "Получен SIGTERM, завершаюсь"
3. Перехватывает SIGUSR1 и выводит "Получен USR1, сбрасываю счётчик"
4. Перехватывает SIGINT и выводит "Ctrl+C нажат, используйте SIGTERM" и НЕ завершается

Запустите скрипт, протестируйте все три сценария.

Задание 3.4 – Systemd служба

Создайте systemd службу для скрипта из задания 3.3:

- Служба должна запускаться от имени обычного пользователя
- При падении должна автоматически перезапускаться
- Логи должны писаться в journald
- Служба должна запускаться после network.target

Включите автозапуск. Проверьте статус, остановите, запустите, проверьте логи через journalctl.

Задание 3.5 – Анализ логов

Используя journalctl и grep, выполните следующее:

1. Найдите все неудачные попытки входа за последние 7 дней
 2. Определите топ-5 пользователей/IP-адресов с наибольшим количеством неудачных попыток
 3. Найдите, какие службы упали (state=failed) за сегодня
 4. Подсчитайте количество перезагрузок системы за последний месяц
- Оформите результаты в виде отчёта с командами и выводом.

Задание 3.6 – Комплексное задание

Создайте bash-скрипт "system_monitor.sh", который:

1. Проверяет загрузку CPU (если >80% – пишет в лог WARNING)
2. Проверяет использование памяти (если >90% – пишет CRITICAL)
3. Проверяет статус служб: ssh, cron (если не running – ALERT)
4. Записывает все сообщения через logger с тегом "sysmon"
5. Настройте запуск скрипта каждые 5 минут через cron
6. Настройте rsyslog для записи сообщений sysmon в отдельный файл

СПРАВОЧНИК КОМАНД

ПОЛЬЗОВАТЕЛИ И ГРУППЫ

useradd -m -s /bin/bash -c "Comment" username	# создать пользователя
usermod -aG groupname username	# добавить в группу
usermod -L username / usermod -U username	# блокировка/разблокировка
userdel -r username	# удалить с домашней директорией
passwd username	# сменить пароль
chage -l username	# просмотр политики пароля
chage -m 1 -M 90 -W 14 -I 30 username	# настроить политику
chage -d 0 username	# принудить сменить пароль
groupadd -g GID groupname	# создать группу
groupdel groupname	# удалить группу

id username	# информация о пользователе
getent passwd / getent group	# просмотр баз данных

ПРАВА ДОСТУПА

chmod 755 file	# числовая запись
chmod u+x,g-w,o= file	# символьная запись
chmod -R 755 directory	# рекурсивно
chmod u+s file	# установить SUID
chmod g+s directory	# установить SGID
chmod +t directory	# установить Sticky bit
chmod 4755 / 2755 / 1755 file	# SUID/SGID/Sticky в числах
chown user:group file	# сменить владельца и группу
chown -R user:group directory	# рекурсивно
chgrp group file	# сменить только группу
umask	# показать текущую umask
umask 027	# установить umask
find / -perm /4000	# найти файлы с SUID
find / -perm -o+w -type f	# найти файлы, доступные всем для записи
find / -nouser -o -nogroup	# найти файлы без владельца

ПРОЦЕССЫ

ps aux	# все процессы
ps auxf	# с деревом
pgrep -la nginx	# найти по имени
jobs -l	# список заданий с PID
fg %2 / bg %2	# перевести задание
kill -15 PID / kill -9 PID	# SIGTERM / SIGKILL
pkill -9 processname	# по имени
killall processname	# по точному имени
nice -n 10 command	# запуск с приоритетом
renice 10 -p PID	# изменить приоритет
nohup command &	# устойчивый к закрытию терминала
disown %1	# отвязать от терминала
top / htop	# мониторинг
top -b -n 1 > snapshot.txt	# снимок состояния

SYSTEMD

```
systemctl status|start|stop|restart|reload service
systemctl enable|disable|enable --now service
systemctl list-units --type=service --state=running
systemctl daemon-reload
systemctl get-default
systemctl set-default multi-user.target
systemctl list-unit-files --type=service
```

ЛОГИ

journalctl -f	# live мониторинг
journalctl -u service -f	# логи службы live
journalctl -p err --since "1 hour ago"	# ошибки за час
journalctl -b	# текущая загрузка
journalctl -b -1	# предыдущая загрузка
journalctl --disk-usage	# размер журнала
journalctl --vacuum-time=30d	# очистка старше 30 дней
tail -f /var/log/syslog	# традиционный syslog
grep "pattern" /var/log/auth.log	# поиск в логах
logger -t myapp "message"	# запись в syslog